Updated FHIR CodeableConcept & Terminology Platform Architecture

Overview

This document defines an enterprise-grade FHIR terminology architecture for managing:

- CodeableConcept
- Coding
- ValueSet
- CodeSystem
- terminology validation
- AI-assisted terminology mapping
- dynamic UI generation
- workflow terminology
- multilingual concepts
- semantic search
- organization-specific terminology extensions

This architecture is designed for:

- FHIR R4/R5 systems
- AI-native healthcare platforms
- Dynamic form generation
- Agentic workflows
- Enterprise healthcare interoperability
- Multi-tenant hospital systems

Core Philosophy

The terminology platform should NOT be treated as:

```
Dropdown management
```

Instead, it should become:

```
Healthcare semantic infrastructure
```

This terminology layer powers:

- FHIR validation
- AI reasoning
- Dynamic UI generation
- Workflow orchestration
- CDS systems
- Analytics
- Interoperability
- NLP extraction
- Semantic search

HIGH LEVEL ARCHITECTURE

```
FHIR Resources
    ↓
CodeableConcept
    ↓
Terminology Platform
    ├── CodeSystems
    ├── ValueSets
    ├── Concepts
    ├── Validation
    ├── Expansion
    ├── AI Mapping
    ├── Semantic Search
    ├── Translation
    ├── Governance
    └── Hierarchies
            ↓
Workflow Engine
            ↓
AI Agents
            ↓
Dynamic UI
```

FHIR TERMINOLOGY CORE CONCEPTS

---

1. CodeSystem

Defines the full vocabulary.

Examples:

- SNOMED CT
- LOINC
- ICD-10
- RxNorm
- HL7 code systems

Example:

```
SNOMED CT contains ALL SNOMED concepts
```

---

2. ValueSet

Subset of codes allowed for a specific use case.

Example:

```
Condition Clinical Status
```

Contains only:

- active
- resolved
- remission
- relapse

---

3. Coding

Single code instance.

Example:

```json
{
  "system": "http://snomed.info/sct",
  "code": "44054006",
  "display": "Diabetes mellitus type 2"
}
```

---

4. CodeableConcept

Collection of codings + optional text.

Example:

```json
{
  "coding": [
    {
      "system": "http://snomed.info/sct",
      "code": "44054006",
      "display": "Diabetes mellitus type 2"
    }
  ],
  "text": "Type 2 Diabetes"
}
```

RECOMMENDED DATABASE ARCHITECTURE

Core Tables

```
terminology_code_system
terminology_value_set
terminology_value_set_concept
terminology_concept
terminology_concept_synonym
terminology_concept_translation
terminology_relationship
terminology_ai_mapping
terminology_field_binding
```

```
terminology_concept_embedding
terminology_audit_log
```

TABLE 1 — terminology_code_system

Defines vocabulary metadata.

```sql
CREATE TABLE terminology_code_system (
    id SERIAL PRIMARY KEY,

    canonical_url VARCHAR NOT NULL UNIQUE,
    name VARCHAR NOT NULL,
    title VARCHAR,
    description TEXT,

    version VARCHAR,
    fhir_version VARCHAR,

    publisher VARCHAR,
    jurisdiction VARCHAR,

    content_mode VARCHAR,
    experimental BOOLEAN DEFAULT FALSE,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Example

```
http://snomed.info/sct
http://loinc.org
http://hl7.org/fhir/sid/icd-10
```

TABLE 2 — terminology_concept

Stores concepts/codes.

```sql
CREATE TABLE terminology_concept (
    id SERIAL PRIMARY KEY,

    code_system_id INTEGER REFERENCES terminology_code_system(id),

    code VARCHAR NOT NULL,
    display VARCHAR NOT NULL,

    definition TEXT,

    active BOOLEAN DEFAULT TRUE,
    deprecated BOOLEAN DEFAULT FALSE,

    parent_concept_id INTEGER NULL REFERENCES terminology_concept(id),

    search_vector tsvector,

    org_id VARCHAR NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    UNIQUE(code_system_id, code, org_id)
);
```

IMPORTANT DESIGN

This table supports:

```
Standard Concepts
+
Organization-Specific Concepts
```

Example

```
SNOMED → Diabetes
Hospital-specific workflow code
Custom insurance categories
```

TABLE 3 — terminology_concept_synonym

Supports NLP and semantic search.

```sql
CREATE TABLE terminology_concept_synonym (
    id SERIAL PRIMARY KEY,

    concept_id INTEGER REFERENCES terminology_concept(id),

    synonym VARCHAR NOT NULL
);
```

Example

```
Heart Attack
Myocardial Infarction
MI
```

TABLE 4 — terminology_concept_translation

Multilingual support.

```sql
CREATE TABLE terminology_concept_translation (
    id SERIAL PRIMARY KEY,

    concept_id INTEGER REFERENCES terminology_concept(id),

    language_code VARCHAR NOT NULL,

```

```
    display VARCHAR NOT NULL
);
```

Example

```
en → Diabetes
hi → मधुमेह
ta → நீரிழிவு
```

TABLE 5 — terminology_relationship

Supports hierarchical and semantic relationships.

```
CREATE TABLE terminology_relationship (
    id SERIAL PRIMARY KEY,

    parent_concept_id INTEGER REFERENCES terminology_concept(id),
    child_concept_id INTEGER REFERENCES terminology_concept(id),

    relationship_type VARCHAR NOT NULL
);
```

Relationship Types

```
is-a
part-of
associated-with
causes
related-to
```

Example

```
Diabetes
    ├── Type 1 Diabetes
    ├── Type 2 Diabetes
    └── Gestational Diabetes
```

TABLE 6 — terminology_value_set

Defines allowed code groups.

```sql
CREATE TABLE terminology_value_set (
    id SERIAL PRIMARY KEY,

    canonical_url VARCHAR NOT NULL UNIQUE,

    name VARCHAR NOT NULL,
    title VARCHAR,

    description TEXT,

    version VARCHAR,
    fhir_version VARCHAR,

    binding_strength VARCHAR NOT NULL,

    experimental BOOLEAN DEFAULT FALSE,

    active BOOLEAN DEFAULT TRUE,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Example

```
Condition Clinical Status
Appointment Status
Observation Category
```

TABLE 7 — terminology_value_set_concept

Maps concepts into ValueSets.

```sql
CREATE TABLE terminology_value_set_concept (
    id SERIAL PRIMARY KEY,

    value_set_id INTEGER REFERENCES terminology_value_set(id),
    concept_id INTEGER REFERENCES terminology_concept(id),

    active BOOLEAN DEFAULT TRUE,

    UNIQUE(value_set_id, concept_id)
);
```

IMPORTANT

A concept can belong to:

```
Multiple ValueSets
```

TABLE 8 — terminology_field_binding

Maps:

```
FHIR Resource + Field
→
ValueSet
```

```sql
CREATE TABLE terminology_field_binding (
    id SERIAL PRIMARY KEY,

    resource_type VARCHAR NOT NULL,
    field_name VARCHAR NOT NULL,

    value_set_id INTEGER REFERENCES terminology_value_set(id),

    binding_strength VARCHAR NOT NULL,

    multiple_allowed BOOLEAN DEFAULT FALSE,

    active BOOLEAN DEFAULT TRUE,

    UNIQUE(resource_type, field_name)
);
```

Example

```
Condition.clinicalStatus
    → condition-clinical

Observation.category
    → observation-category
```

TABLE 9 — terminology_ai_mapping

AI/NLP phrase mappings.

```sql
CREATE TABLE terminology_ai_mapping (
    id SERIAL PRIMARY KEY,

    phrase TEXT NOT NULL,

    concept_id INTEGER REFERENCES terminology_concept(id),

    confidence FLOAT,

    source VARCHAR,
```

```
    created_at TIMESTAMP DEFAULT NOW()
);
```

Example

```
"high sugar"
    → Diabetes mellitus

"heart attack"
    → Myocardial infarction
```

TABLE 10 — terminology_concept_embedding

Stores vector embeddings.

```
CREATE TABLE terminology_concept_embedding (
    id SERIAL PRIMARY KEY,

    concept_id INTEGER REFERENCES terminology_concept(id),

    embedding vector(1536)
);
```

WHY IMPORTANT

Enables:

- semantic search
- AI retrieval
- similarity matching
- intelligent autocomplete
- symptom matching

TABLE 11 — terminology_audit_log

Terminology governance.

```
CREATE TABLE terminology_audit_log (
    id SERIAL PRIMARY KEY,

    action VARCHAR NOT NULL,

    concept_id INTEGER NULL REFERENCES terminology_concept(id),
    value_set_id INTEGER NULL REFERENCES terminology_value_set(id),

    performed_by VARCHAR,

    old_value JSONB,
    new_value JSONB,

    created_at TIMESTAMP DEFAULT NOW()
);
```

WHY IMPORTANT

Tracks:

- code changes
- display changes
- AI updates
- governance history

RECOMMENDED API ARCHITECTURE

Terminology APIs

```
GET  /terminology/code-systems
GET  /terminology/value-sets
GET  /terminology/fields
GET  /terminology/concepts
GET  /terminology/search
POST /terminology/validate
```

```
POST /terminology/expand
POST /terminology/lookup
POST /terminology/lookup-batch
POST /terminology/concepts
```

IMPORTANT ENDPOINTS

1. Get Concepts for Resource Field

```
GET /terminology/concepts?resource=Condition&field=clinicalStatus
```

---

2. Terminology Validation

```
POST /terminology/validate
```

Input:

```
{
  "resource": "Condition",
  "field": "clinicalStatus",
  "system": "http://terminology.hl7.org/CodeSystem/condition-clinical",
  "code": "active"
}
```

---

3. Semantic Search

```
GET /terminology/search?q=heart attack
```

Returns:

```
Myocardial infarction
```

---

4. ValueSet Expansion

```
GET /terminology/value-sets/{id}/expand
```

Supports:

- pagination
- filtering
- search
- hierarchy traversal

FHIR BINDING STRENGTHS

Binding	Meaning
required	Only allowed codes
preferred	Prefer codes but allow others
extensible	Extendable
example	Suggestions only

FRONTEND ARCHITECTURE

Dynamic Form Flow

```
Frontend Opens Form
    ↓
GET /terminology/fields
    ↓
GET /terminology/concepts
    ↓
Render Dynamic Dropdowns
```

Recommended Frontend Behavior

Always send:

```json
{
  "system": "http://snomed.info/sct",
  "code": "44054006",
  "display": "Diabetes mellitus type 2"
}
```

Never send only:

```json
{
  "code": "44054006"
}
```

AI INTEGRATION ARCHITECTURE

AI Workflow

```
Clinical Text
    ↓
AI Extraction
    ↓
Terminology Semantic Search
    ↓
Suggested Concepts
    ↓
Human Review
    ↓
FHIR Resource Generation
```

Example

```
Patient says:
"I have high sugar"
```

AI maps to:

```
SNOMED → Diabetes mellitus
```

RECOMMENDED CACHING STRATEGY

Cache Layers

```
Browser Cache
↓
CDN Cache
↓
Redis Cache
↓
Postgres
```

Suggested TTL

```
Terminology TTL = 24 hours
```

Terminology changes slowly.

EVENT-DRIVEN TERMINOLOGY

Recommended Events

```
terminology.concept.created
terminology.concept.updated
terminology.value_set.updated
terminology.embedding.generated
```

WHY IMPORTANT

Allows:

- cache invalidation
- AI embedding regeneration
- frontend live refresh
- analytics updates

MULTI-TENANT SUPPORT

Supports:

```
Global Standard Codes
+
Organization-Specific Codes
```

Example

```
Hospital-specific triage categories
Custom billing classifications
Insurance workflow tags
```

RECOMMENDED TERMINOLOGY MODULE STRUCTURE

```
terminology/
    registry/
    validation/
    expansion/
    search/
    ai/
    import/
    governance/
    translation/
    embedding/
```

RECOMMENDED PYTHON PROJECT STRUCTURE

```
app/
├── terminology/
│   ├── models/
│   ├── services/
│   ├── repositories/
│   ├── routers/
│   ├── validators/
│   ├── ai/
│   ├── embeddings/
│   ├── search/
│   ├── cache/
│   └── governance/
```

RECOMMENDED ADDITIONAL FHIR RESOURCES

Strongly recommended:

Resource	Purpose
CodeSystem	Vocabulary definition
ValueSet	Allowed code groups

Resource	Purpose
ConceptMap	Cross-system mapping
NamingSystem	Identifier systems
StructureDefinition	Dynamic forms

CONCEPTMAP SUPPORT (IMPORTANT)

FHIR supports:

```
ConceptMap
```

Used for:

```
ICD-10 ↔ SNOMED
LOINC ↔ Local Lab Codes
```

Recommended Table

```sql
CREATE TABLE terminology_concept_map (
    id SERIAL PRIMARY KEY,

    source_concept_id INTEGER REFERENCES terminology_concept(id),
    target_concept_id INTEGER REFERENCES terminology_concept(id),

    mapping_type VARCHAR,

    confidence FLOAT
);
```

AI + TERMINOLOGY FUTURE

This architecture enables:

- AI coding agents

- semantic clinical search
- NLP extraction
- CDS systems
- intelligent form generation
- auto-suggestions
- workflow automation
- analytics
- interoperability

RECOMMENDED TECHNOLOGIES

Layer	Recommendation
DB	PostgreSQL
Vector Search	pgvector
FullText Search	PostgreSQL tsvector
Cache	Redis
API	FastAPI
Queue	Kafka / RabbitMQ
AI	Embedding Models

BIGGEST ARCHITECTURAL PRINCIPLE

Treat terminology as:

```
Healthcare Semantic Platform
```

NOT:

```
Simple dropdown management
```

FINAL RECOMMENDED ARCHITECTURE

```
FHIR Resources
    ↓
Terminology Platform
    ├── Validation
    ├── Expansion
    ├── Semantic Search
    ├── AI Mapping
    ├── Embeddings
    ├── Translation
    ├── Governance
    └── Hierarchies
            ↓
Workflow Engine
            ↓
AI Agents
            ↓
Dynamic Forms
            ↓
Clinical Decision Support
```

FINAL IMPLEMENTATION PRIORITIES

Recommended order:

1. CodeSystem
2. ValueSet
3. Concept
4. Field Bindings
5. Validation Engine
6. Search APIs
7. Semantic Search
8. AI Mapping
9. Embeddings
10. Governance
11. Translation
12. ConceptMap

FINAL SUMMARY

Your original architecture direction is already very good.

The optimized enterprise-grade architecture should add:

- CodeSystem support
- semantic search
- AI mappings
- vector embeddings
- hierarchy traversal
- multilingual support
- governance
- validation engine
- ConceptMap support
- event-driven terminology updates

This transforms terminology into:

```
AI + Interoperability + Workflow + Semantic Infrastructure Layer
```

instead of merely:

```
dropdown lookup tables
```

This architecture scales extremely well for:

- AI-native EMRs
- enterprise healthcare systems
- dynamic form systems
- workflow orchestration
- CDS systems
- healthcare automation
- multi-agent healthcare AI platforms